

Task 2 Development Documentation

Greenfield Local Hub (GLH) Digital Solution

Frontend: React & TypeScript | Backend: Node.js & Express | Database: PostgreSQL

General Information

This documentation covers the development of a digital solution built for Greenfield Local Hub (GLH), a cooperative of local farmers and food producers. The solution was developed to serve both customers and producers, addressing all key requirements set out in the client brief.

The frontend was built by using React and TypeScript, this was done while utilising the reusable components from end to end. The backend was developed with Node.js and Express while being connected to a PostgreSQL database. All the navigation bar links, footer links, and profile/account area links are perfectly working.

The prototype includes the following features:

- Customer and producer account management with access based on their roles
- Product browsing with clear pricing and availability of information
- Order placement supporting both collection and delivery options
- Order history with a reorder functionality
- A producer dashboard for managing stock and product listings
- A loyalty discount scheme providing discounts and special offers
- Accessibility options including larger text, high contrast, and reduced motion

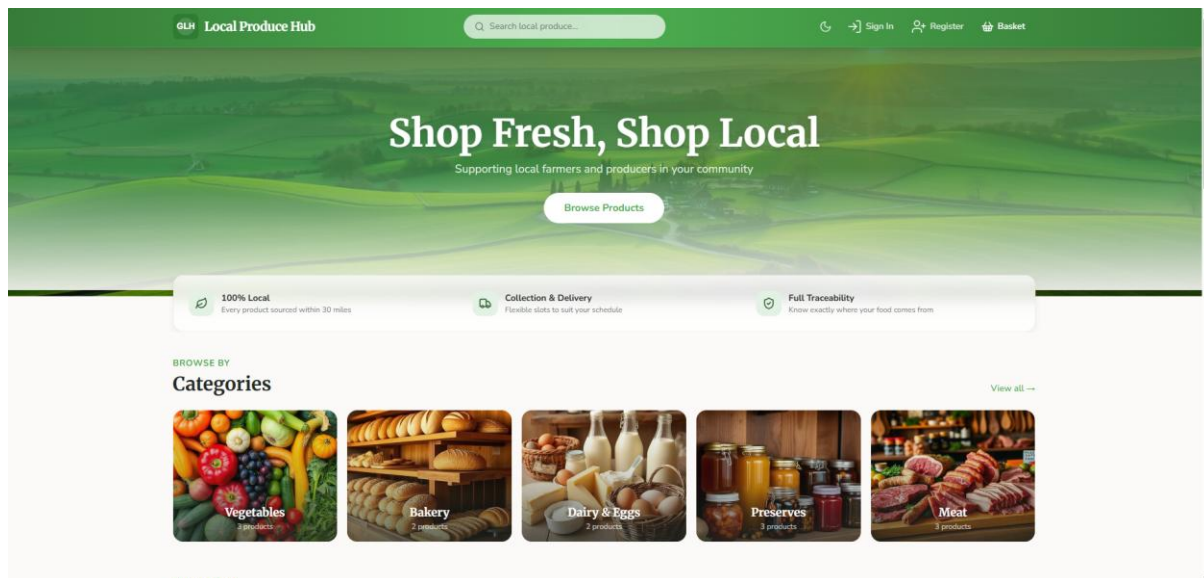
Home Page

This is the home page of the prototype. It showcases the main navigation bar, footer, and the overall branding style for GLH. The home link in the navigation bar has been highlighted allowing the user to clearly identify which page they are currently navigating on.

I gave this page a clean and focused look introducing GLH, communicating the value of buying local produce, and guiding users towards the products page and account area. The layout is well responsive and adjusts well across different screen sizes.

Unlike a traditional static HTML site, the page structure here is built using React components. The main files involved are `src/pages/Index.tsx` for the page itself, `src/components/Navbar.tsx` for the navigation, and `src/components/Footer.tsx` for the footer.

Screenshot Home Page



Screenshot Home Page Code (src/pages/Index.tsx)

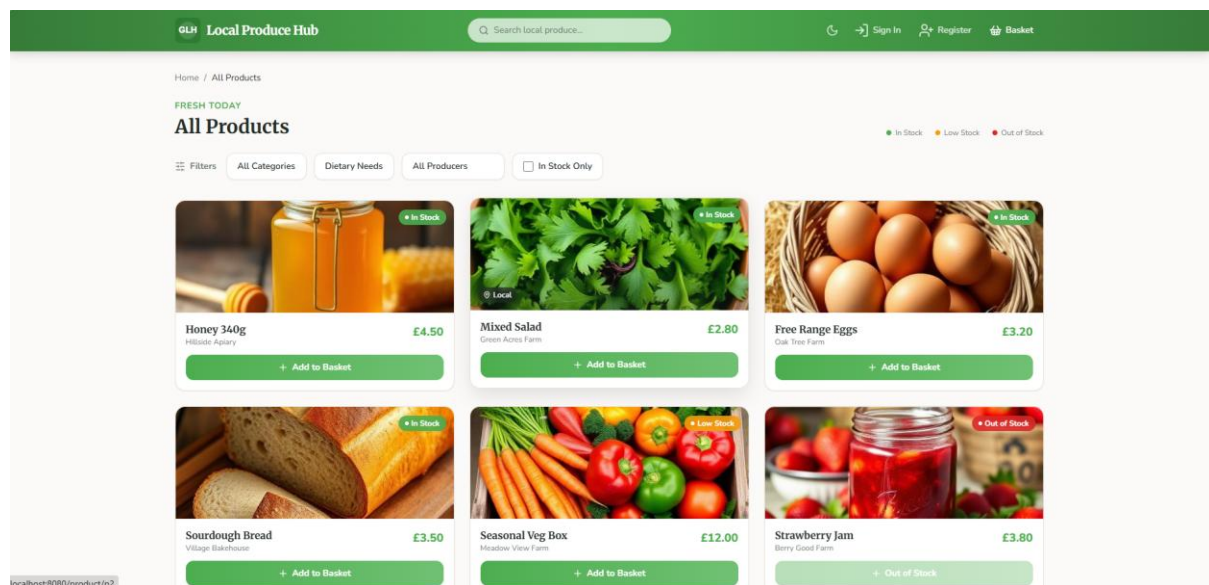
```
1 import { useState, useMemo, useRef } from "react";
2 import { Link, useNavigate } from "react-router-dom";
3 import Navbar from "@components/Navbar";
4 import HeroBanner from "@components/HeroBanner";
5 import LoyaltyBanner from "@components/LoyaltyBanner";
6 import CategoryGrid from "@components/CategoryGrid";
7 import FilterBar from "@components/FilterBar";
8 import ProductCard from "@components/ProductCard";
9 import HowItWorks from "@components/HowItWorks";
10 import ProducerSpotlight from "@components/ProducerSpotlight";
11 import Footer from "@components/Footer";
12 import { products } from "@data/products";
13 import { useAuth } from "@context/AuthContext";
14
15 // Home page: handles filters/search and shows featured products.
16 export default function Index() {
17   const [search, setSearch] = useState("");
18   const [category, setCategory] = useState("All");
19   const [dietary, setDietary] = useState("All");
20   const [producer, setProducer] = useState("All");
21   const [inStockOnly, setInStockOnly] = useState(false);
22   const productsRef = useRef<HTMLDivElement>(null);
23   const { user } = useAuth();
24
25   const filtered = useMemo(() => {
26     // Keep all filter logic in one place so it is easy to adjust later.
27     return products.filter((p) => {
28       if (search) {
29         const q = search.toLowerCase();
30         const matchesName = p.name.toLowerCase().includes(q);
31         const matchesProducer = p.producer.toLowerCase().includes(q);
32         const matchesCategory = p.category.toLowerCase().includes(q);
33         const matchesDescription = p.description?.toLowerCase().includes(q);
34         if (!matchesName && !matchesProducer && !matchesCategory && !matchesDescription) return false;
35       }
36       if (category !== "All" && p.category !== category) return false;
37       if (dietary !== "All" && !p.dietary.includes(dietary)) return false;
38       if (producer !== "All" && p.producer !== producer) return false;
39       if (inStockOnly && p.stock === 0) return false;
40       return true;
41     });
42   }, [search, category, dietary, producer, inStockOnly]);
43
44   const navigate = useNavigate();
45 }
```

Products Page

The products page lets customers browse all available produce in a clear card layout. Each card displays the product name, the producer it comes from, its price, and current availability. The aim was to keep this page readable and transparent, which directly addresses the client requirement for visible pricing and availability information.

This page is implemented in `src/pages/Products.tsx`. This makes use of reusable UI components and consistent styling in line with the rest of the prototype.

Screenshot Products Page



Screenshot Products Page Code (src/pages/Products.tsx)

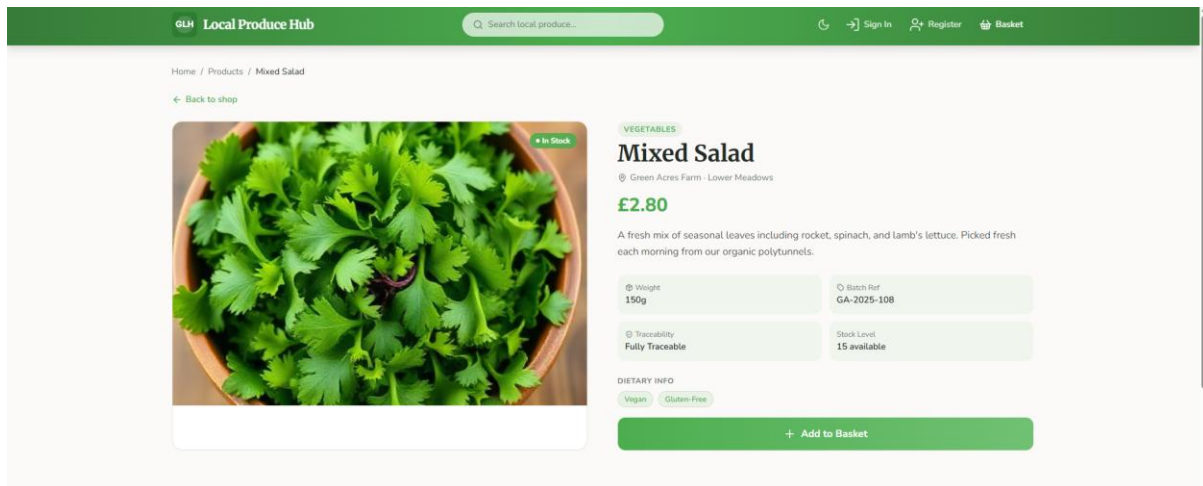
```
1 import { useState, useMemo, useEffect } from "react";
2 import { Link, useSearchParams } from "react-router-dom";
3 import Navbar from "@components/Navbar";
4 import FilterBar from "@components/FilterBar";
5 import ProductCard from "@components/ProductCard";
6 import Footer from "@components/Footer";
7 import { products } from "@data/products";
8
9 // Full products page: same filter logic as home, but shows all matching items.
10 export default function Products() {
11   const [searchParams] = useSearchParams();
12   const [search, setSearch] = useState(searchParams.get("search") || "");
13   const [category, setCategory] = useState("All");
14   const [dietary, setDietary] = useState("All");
15   const [producer, setProducer] = useState("All");
16   const [inStockOnly, setInStockOnly] = useState(false);
17
18   useEffect(() => {
19     const q = searchParams.get("search");
20     if (q) setSearch(q);
21   }, [searchParams]);
22
23   const filtered = useMemo(() => {
24     // Filter chain is grouped here for readability and reuse.
25     return products.filter((p) => {
26       if (search) {
27         const q = search.toLowerCase();
28         const matchesName = p.name.toLowerCase().includes(q);
29         const matchesProducer = p.producer.toLowerCase().includes(q);
30         const matchesCategory = p.category.toLowerCase().includes(q);
31         const matchesDescription = p.description?.toLowerCase().includes(q);
32         if (!matchesName && !matchesProducer && !matchesCategory && !matchesDescription) return false;
33       }
34       if (category !== "All" && p.category !== category) return false;
35       if (dietary !== "All" && !p.dietary.includes(dietary)) return false;
36       if (producer !== "All" && p.producer !== producer) return false;
37       if (inStockOnly && p.stock === 0) return false;
38       return true;
39     });
40   }, [search, category, dietary, producer, inStockOnly]);
41
42   const handleSearch = (query: string) => {
43     setSearch(query);
44   };
45
46   return (
47     <div className="flex flex-col min-h-screen">
```

Product Detail Page

After selecting a product off the products page, the user will be redirected to the product detail page which displays the full details regarding their selected product. This contains information including it's price, current stock level, and all other important information. Customers will also get the option to add the item to their basket.

This page is implemented in `src/pages/ProductDetail.tsx`.

Screenshot Product Detail Page



Screenshot Product Detail Page Code (src/pages/ProductDetail.tsx)

```
1 import { useParams, Link } from "react-router-dom";
2 import { products, getStockStatus } from "@data/products";
3 import NavBar from "@components/NavBar";
4 import Footer from "@components/Footer";
5 import StockBadge from "@components/StockBadge";
6 import { useCart } from "@context/CartContext";
7 import { toast } from "sonner";
8 import { ArrowLeft, Plus, HaptPin, ShieldCheck, Tag, Package, AlertTriangle } from "lucide-react";
9
10 // Product detail page: detailed info + add-to-basket action.
11 export default function ProductDetail() {
12   const { id } = useParams<{ id: string }>();
13   const { addToCart } = useCart();
14   const product = products.find((p) => p.id === id);
15
16   if (!product) {
17     return (
18       <div className="flex flex-col min-h-screen">
19         <NavBar />
20         <main (alias) const Link: ForwardRefExoticComponent<LinkProps & RefAttributes<HTMLAnchorElement>>
21           <div>
22             <Link />
23             < The public API for rendering a history-aware <a> .
24             <Link to="/" className="text-primary font-semibold hover:underline"> Back to shop</Link>
25           </div>
26         </main>
27         <Footer />
28       </div>
29     );
30   }
31
32   const status = getStockStatus(product.stock);
33   const relatedProducts = products.filter(
34     (p) => p.category === product.category && p.id !== product.id
35   ).slice(0, 3);
36
37   const handleAdd = () => {
38     // Block add if stock is empty, otherwise add to basket.
39     if (status === "out-of-stock") return;
40     addToCart(product);
41     toast.success(`${product.name} added to basket`);
42   };
43
44   return (
45     <div className="flex flex-col min-h-screen">
46       <NavBar />
47       <main className="flex-1 container mx-auto px-4 py-8">
```

Register Page

The register page allows new users to create an account as either a customer or a producer. This dual role registration was an important requirement as the GLH solution needs to serve both the groups differently.

To ensure that the registration process is reliable, I put in place several layers of input validation. The verifications include matching password confirmation, required fields, a correctly formatted email, and a backend check which is in place to prevent the same email addresses from being registered twice. If any of these validations are unsuccessful, the user will receive a clear error message, and therefore no account is created.

This page is implemented in `src/pages/Account.tsx` operating in register mode. The backend logic runs through Node.js and Express rather than PHP, which is not used anywhere in this project.

Screenshot Register Page

Local Produce Hub Back to Store

Create Account

Full Name

Email

Password

Minimum 8 characters

Account type

☒ **Customer**
Shop and collect loyalty points on purchases.

☐ **Producer**
Shop like a customer plus access the producer dashboard to manage listings.

☐ I agree to GLH storing my data in accordance with the [Privacy Policy](#)

Create Account

Already have an account? [Sign in](#)

Local Produce Hub QUICK LINKS LEGAL

Screenshot Validation Error Examples

Local Produce Hub

Back to Store

Create Account

Full Name

mohammed abdi

Email

mohammed.com

Please include an '@' in the email address. 'mohammed.com' is missing an '@'.

Minimum 8 characters

Account type

☒ Customer

Shop and collect loyalty points on purchases.

☐ Producer

Shop like a customer plus access the producer dashboard to manage listings.

☒ I agree to GLH storing my data in accordance with the [Privacy Policy](#)

Create Account

Already have an account? [Sign in](#)

Local Produce Hub

QUICK LINKS

LEGAL

Screenshot Register Page Code (src/pages/Account.tsx)

```
1 import { useState, useEffect } from "react";
2 import { Link, useSearchParams, useNavigate } from "react-router-dom";
3 import Footer from "@components/Footer";
4 import { toast } from "sonner";
5 import { login, signup, type AccountRole } from "@lib/api/auth";
6 import { useAuth } from "@context/AuthContext";
7
8 // Account page combines sign in + sign up in one place.
9 ~ function homePathForRole(role: AccountRole) {
10   return role === "producer" ? "/dashboard" : "/";
11 }
12
13 ~ export default function Account() {
14   const [searchParams] = useSearchParams();
15   const navigate = useNavigate();
16   const { user, refreshSession } = useAuth();
17   const [isLogin, setIsLogin] = useState(true);
18   const [email, setEmail] = useState("");
19   const [password, setPassword] = useState("");
20   const [name, setName] = useState("");
21   ~ const [accountRole, setAccountRole] = useState<AccountRole>(() => {
22     searchParams.get("role") === "producer" ? "producer" : "customer",
23   });
24   const [consent, setConsent] = useState(false);
25   const [submitting, setSubmitting] = useState(false);
26
27   ~ useEffect(() => {
28     if (searchParams.get("mode") === "register") {
29       setIsLogin(false);
30     }
31   }, [searchParams]);
32
33   ~ useEffect(() => {
34     if (searchParams.get("role") === "producer") {
35       setAccountRole("producer");
36     }
37   }, [searchParams]);
38
39   const redirect = searchParams.get("redirect");
40
41   // Redirect if already logged in
42   ~ useEffect(() => {
43     if (user) {
44       const dest = redirect ? `/${redirect}` : homePathForRole(user.role);
45       navigate(dest, { replace: true });
46     }
47   }, [user, navigate, redirect]);
```

Login Page

The login page is designed to allow existing users to sign into their accounts securely. When a user submits their log in credentials, they are checked with the database. If they match, then the user is signed in and redirected to the appropriate area depending on their role. Customers go to the customer area and producers go to their dashboard.

If the login details are incorrect, then the user is displayed a clear error message and is not granted access to the website. This page is also implemented in `src/pages/Account.tsx`, this time in login mode. As with the register page, the backend is handled through Node.js and Express.

Screenshot Login Page

The screenshot shows the 'Sign In' page of the Local Produce Hub. The page has a green header with 'Local Produce Hub' on the left and 'Back to Store' on the right. The main content area is white and contains a 'Sign In' form. The form has a title 'Sign In' in green, followed by 'Email' and 'Password' labels. The email input field contains 'hello@gmail.com'. The password input field is masked with dots and has a note 'Minimum 8 characters' below it. A green 'Sign In' button is at the bottom of the form. Below the button is a link: 'Don't have an account? [Create one](#)'. The footer is green and contains three columns: 'Local Produce Hub' with a description, 'QUICK LINKS' with links to 'About GLH', 'Contact Us', and 'Become a producer', and 'LEGAL' with links to 'Privacy Policy' and 'Accessibility Statement'. At the bottom of the footer is a copyright notice: '© 2025 Greenfield Local Hub. All rights reserved. | Farm-to-fork traceability in every order.'

Screenshot Login Error

This screenshot shows the 'Sign In' page after an incorrect login attempt. The layout is identical to the previous screenshot, but the email input field now contains 'hello@gmail.com' and the password field is masked with dots. A red error message box is visible at the bottom right of the page, containing the text 'Invalid email or password.' The footer is the same as in the previous screenshot.

Screenshot Login Page Code (src/pages/Account.tsx)

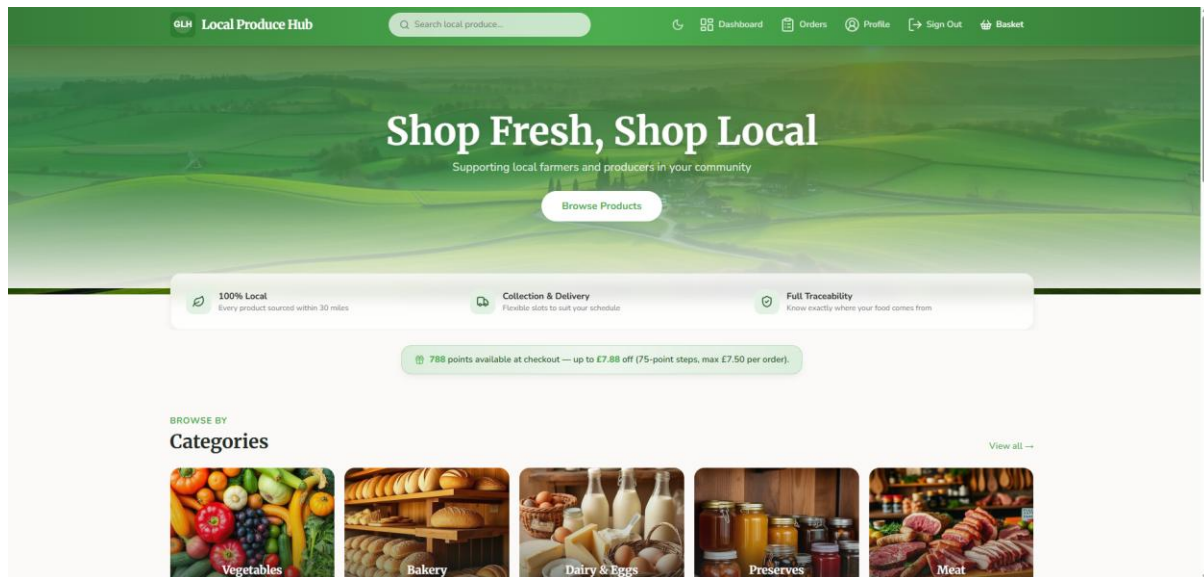
```
1 import { useState, useEffect } from "react";
2 import { Link, useSearchParams, useNavigate } from "react-router-dom";
3 import Footer from "@components/Footer";
4 import { toast } from "sonner";
5 import { login, signUp, type AccountRole } from "@lib/api/auth";
6 import { useAuth } from "@context/AuthContext";
7
8 // Account page combines sign in + sign up in one place.
9 function homePathForRole(role: AccountRole) {
10   return role === "producer" ? "/dashboard" : "/";
11 }
12
13 export default function Account() {
14   const [searchParams] = useSearchParams();
15   const navigate = useNavigate();
16   const { user, refreshSession } = useAuth();
17   const [isLoading, setIsLoading] = useState(true);
18   const [email, setEmail] = useState("");
19   const [password, setPassword] = useState("");
20   const [name, setName] = useState("");
21   const [accountRole, setAccountRole] = useState<AccountRole>(() =>
22     searchParams.get("role") === "producer" ? "producer" : "customer",
23   );
24   const [consent, setConsent] = useState(false);
25   const [submitting, setSubmitting] = useState(false);
26
27   useEffect(() => {
28     if (searchParams.get("mode") === "register") {
29       setIsLoading(false);
30     }
31   }, [searchParams]);
32
33   useEffect(() => {
34     if (searchParams.get("role") === "producer") {
35       setAccountRole("producer");
36     }
37   }, [searchParams]);
38
39   const redirect = searchParams.get("redirect");
40
41   // Redirect if already logged in
42   useEffect(() => {
43     if (user) {
44       const dest = redirect ? `/${redirect}` : homePathForRole(user.role);
45       navigate(dest, { replace: true });
46     }
47   }, [user, navigate, redirect]);
```

Profile Page

Once the user is logged in, they can access their profile and account area through the profile button in the top right corner of the navigation bar. The account flow is fully integrated, meaning the experience adapts depending on whether the logged in user is a customer or a producer.

As with the rest of the backend in this project, this functionality is handled through Node.js and Express.

Screenshot Profile Page



Profile Settings Page

The profile settings page has been made so it allows users access to a range of controls reflecting the account from a single location. These changes include updating profile details, changing their password, managing payment and banking information, setting accessibility preferences, and accessing their order history.

I also made this page specific in its behaviour. For customer accounts, the delivery tab is hidden as it is not relevant to them. Producer accounts have access to the full settings panel, including specific features only for producers such as banking details for payments.

This page is implemented across `src/pages/AccountSettingsPage.tsx` and `src/components/dashboard/AccountSettings.tsx`.

Screenshot Profile Settings Page (Customer View)

The screenshot shows the 'Local Produce Hub' interface for a customer. The top navigation bar is green with the logo 'LPH Local Produce Hub', a search bar, and links for Orders, Profile, Sign Out, and Basket. The main content area has a light beige background. At the top, there's an 'Order History' section with a link to 'Open previous orders'. Below this is the 'Account Settings' section, which has a horizontal tab bar with 'Profile' (selected), Password, Branding, Payment, Notifications, and Accessibility. The 'Profile Information' form includes fields for Account Name, Contact Name (placeholder: 'Your full name'), Email (placeholder: 'helloo@gmail.com'), Phone (placeholder: '07xxx xxxxxx'), Location (placeholder: 'e.g. Gloucestershire'), and a Bio (placeholder: 'Tell customers about your produce and story...').

Screenshot Profile Settings Page (Producer View)

The screenshot shows the 'Local Produce Hub' interface for a producer. The top navigation bar is green with the logo 'LPH Local Produce Hub', a search bar, and links for Dashboard, Orders, Profile, Sign Out, and Basket. The main content area has a light beige background. At the top, there's an 'Order History' section with a link to 'Open previous orders'. Below this is the 'Account Settings' section, which has a horizontal tab bar with 'Profile' (selected), Password, Branding, Payment, Notifications, Delivery, and Accessibility. The 'Profile Information' form includes fields for Account Name (placeholder: 'Hillside Apiary'), Contact Name (placeholder: 'Your full name'), Email (placeholder: 'hossensiffat@gmail.com'), Phone (placeholder: '07xxx xxxxxx'), Location (placeholder: 'e.g. Gloucestershire'), Bio (placeholder: 'Tell customers about your produce and story...'), and Website (placeholder: 'https://yourfarm.co.uk').

Screenshot Profile Settings Code

```
src > pages > AccountSettingsPage.jsx ...
1 import Navbar from "@components/Navbar";
2 import Footer from "@components/Footer";
3 import AccountSettings from "@components/dashboard/AccountSettings";
4 import { useAuth } from "@context/AuthContext";
5 import { Link } from "react-router-dom";
6
7 // Wrapper page for account settings plus quick links.
8 export default function AccountSettingsPage() {
9   const { user } = useAuth();
10
11   return (
12     <div className="flex min-h-screen flex-col">
13       <Navbar />
14       <main className="container mx-auto flex-1 px-4 py-8">
15         <section className="mb-6 rounded-xl border border-border bg-card p-5">
16           { /* Quick shortcut so users can jump to order history page. */ }
17           <h2 className="mb-2 text-xl font-bold font-serif">Order History</h2>
18           <p className="mb-3 text-sm text-muted-foreground">
19             View your previous orders and reorder items with one click.
20           </p>
21           <Link to="/account/orders" className="text-sm font-semibold text-primary hover:underline">
22             Open previous orders
23           </Link>
24         </section>
25         <AccountSettings userEmail={user?.email ?? ""} userRole={user?.role} />
26       </main>
27       <Footer />
28     </div>
29   );
30 }
31
32
```

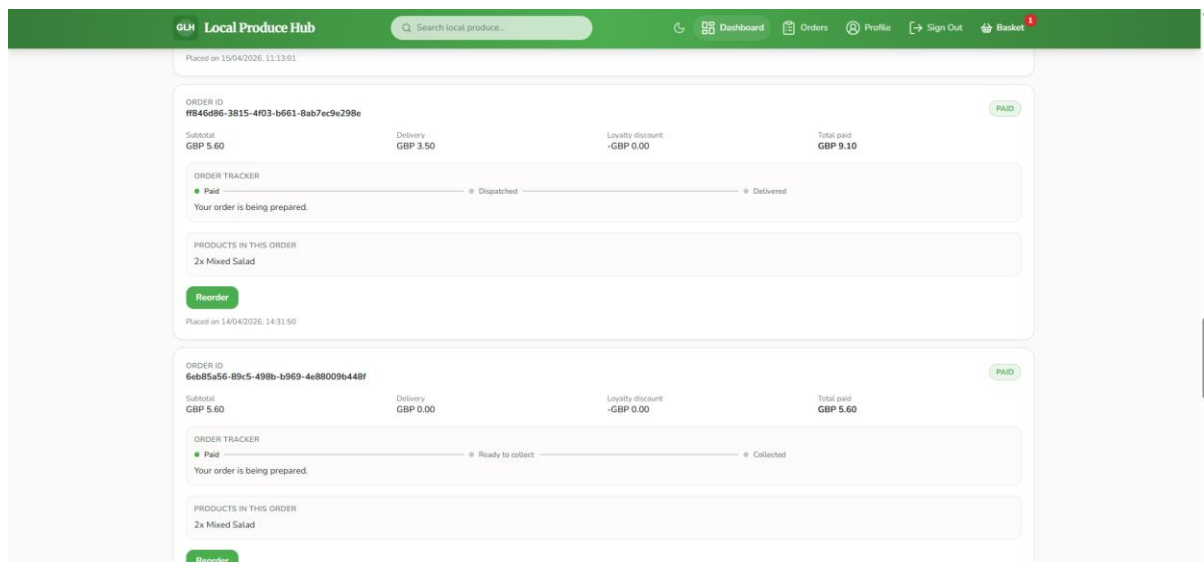
Orders Page / Order History

The order history page was created to allow users to review all of their previous orders. It shows the order ID, the names and the number of items ordered, and a reorder button making it easy to place the same order again. There is also a status tracker displayed for each order.

The tracker acts differently depending on the chosen method of fulfilment type picked at checkout. Delivery orders show status updates such as “Dispatched” and “Delivered”, whereas collection orders show “Ready to Collect” and “Collected”. This feature was important to implement correctly so that the tracker is efficient and accurate for different order types.

This page is implemented in `src/pages/PreviousOrders.tsx`.

Screenshot Order History Page



Screenshot Order Tracker (Delivery vs Collection)

ORDER ID
ff846d86-3815-4f03-b661-8ab7ec9e298e

PAID

Subtotal
GBP 5.60

Delivery
GBP 3.50

Loyalty discount
-GBP 0.00

Total paid
GBP 9.10

ORDER TRACKER

Paid

Dispatched

Delivered

Your order is being prepared.

PRODUCTS IN THIS ORDER

2x Mixed Salad

Reorder

Placed on 14/04/2026, 14:31:50

ORDER ID
6eb85a56-89c5-498b-b969-4e88009b448f

PAID

Subtotal
GBP 5.60

Delivery
GBP 0.00

Loyalty discount
-GBP 0.00

Total paid
GBP 5.60

ORDER TRACKER

Paid

Ready to collect

Collected

Your order is being prepared.

PRODUCTS IN THIS ORDER

2x Mixed Salad

Reorder

Placed on 14/04/2026, 14:19:38

Screenshot Order History Code (src/pages/PreviousOrders.tsx)

```
src > pages > @ PreviousOrders > ...
1 import { useEffect, useId, useImperativeHandle, useState } from "react";
2 import { Link, useNavigate } from "react-router-dom";
3 import { PackageSearch } from "lucide-react";
4 import Navbar from "@components/Navbar";
5 import Footer from "@components/Footer";
6 import { getOrders, type UserOrder } from "@lib/api/orders";
7 import { products } from "@data/products";
8 import { useCart, type CartItem } from "@context/CartContext";
9 import { toast } from "sonner";
10
11 // Convert stored pence values to pounds for display.
12 function pounds(pence: number) {
13   return (pence / 100).toFixed(2);
14 }
15
16 function statusClass(status: UserOrder["status"]) {
17   if (status === "paid") return "bg-success/10 text-success border-success/30";
18   if (status === "refunded") return "bg-warning/10 text-warning border-warning/30";
19   if (status === "cancelled") return "bg-destructive/10 text-destructive border-destructive/30";
20   if (status === "ready") return "bg-primary/10 text-primary border-primary/30";
21   if (status === "dispatched") return "bg-indigo-500/10 text-indigo-600 border-indigo-500/30";
22   return "bg-muted text-muted-foreground border-border";
23 }
24
25 function statusStep(status: UserOrder["status"]): 0 | 3 | 2 | 1 {
26   if (status === "delivered" || status === "collected") return 3;
27   if (status === "dispatched" || status === "ready") return 2;
28   if (status === "paid") return 1;
29   return 0;
30 }
31
32 function trackingMessage(order: UserOrder) {
33   const { status, fulfillmentType } = order;
34   if (status === "ready" && fulfillmentType === "collection") return "Your order is ready to collect.";
35   if (status === "collected") return "Your collection order has been collected.";
36   if (status === "dispatched" && fulfillmentType === "delivery") return "Your order has been dispatched.";
37   if (status === "delivered") return "Your delivery order has been delivered.";
38   if (status === "paid") return "Your order is being prepared.";
39   if (status === "cancelled") return "This order was cancelled.";
40   if (status === "refunded") return "This order was refunded.";
41   return "Tracking status unavailable.";
42 }
43
44 function trackerLabels(fulfillmentType: UserOrder["fulfillmentType"]) {
45   return fulfillmentType === "delivery"
46     ? ["Paid", "Dispatched", "Delivered"]
47     : ["Paid", "Ready to collect", "Collected"];
48 }
```

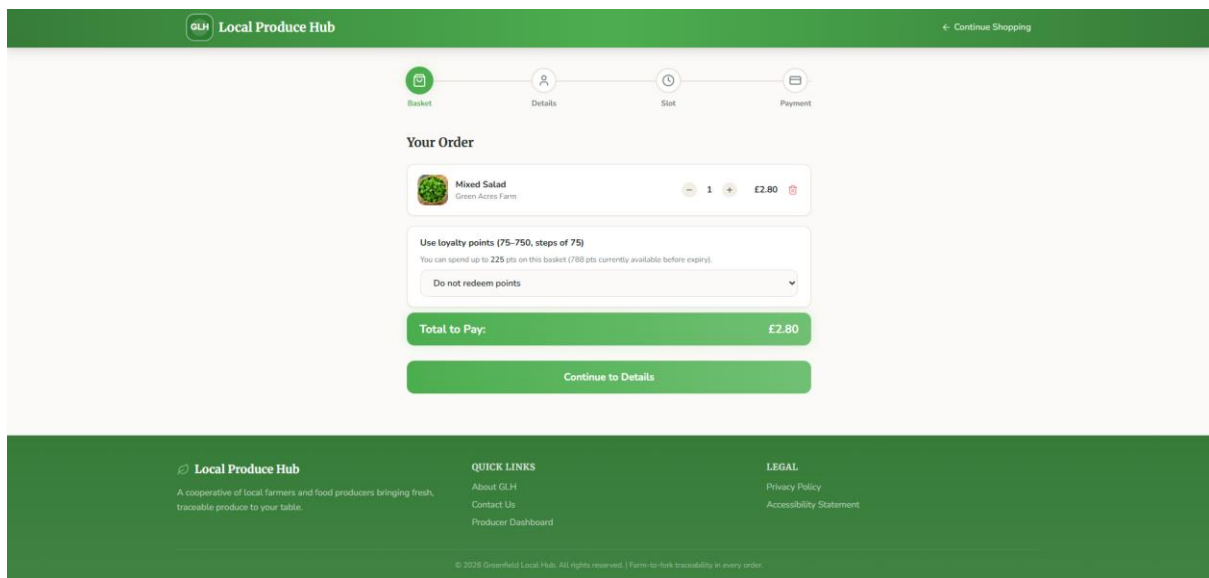
Checkout Page

The checkout process covers several steps including: reviewing the basket, entering customer details, selecting a method to receive the product (collection or delivery), choosing a time slot, making a payment, optionally redeeming loyalty points for a discount, and to finally receiving an order confirmation.

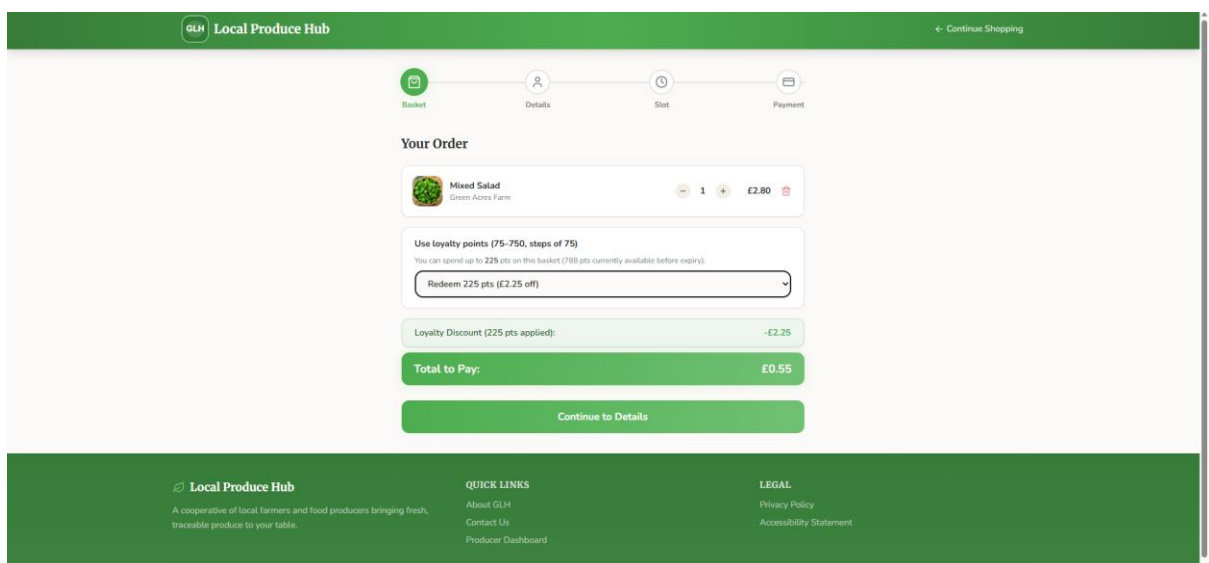
One of the key improvements made during development was ensuring the method of receiving the order and the ordered item lines are correctly stored at the point of order. This was important because it directly impacts how the order history and tracker display later on. Previously, the tracker was showing incorrect statuses due to this not being saved properly.

This page is implemented in `src/pages/Checkout.tsx`.

Screenshot Checkout Flow



Screenshot Loyalty Redemption at Checkout



Screenshot Checkout Code (src/pages/Checkout.tsx)

```
src > pages > @ Checkout.tsx > ...
1  import { useState, useEffect, useMemo } from "react";
2  import { Link, useNavigate } from "react-router-dom";
3  import { useCart } from "@context/CartContext";
4  import type { CartItem } from "@context/CartContext";
5  import { useAuth } from "@context/AuthContext";
6  import Footer from "@components/footer";
7  import { toast } from "sonner";
8  import { Trash2, Minus, Plus, ShoppingBag, Clock, CreditCard, Check, ArrowLeft, User, UserPlus, MapPin, Phone, Mail } from "lucide-react";
9  import {
10    formatAddress,
11    loadCheckoutProfile,
12    saveCheckoutProfile,
13    type SavedDeliveryAddress,
14  } from "@lib/checkoutProfile";
15  import { postOrderCheckout } from "@lib/api/loyalty";
16  import { redemptionOptions } from "@lib/loyaltyRules";
17
18  const logo = "/favicon-ghh.svg";
19  const DELIVERY_FEE_PENCE = 350;
20
21  const slots = [
22    { time: "09:00", status: "available" as const },
23    { time: "10:00", status: "available" as const },
24    { time: "11:00", status: "full" as const },
25    { time: "14:00", status: "available" as const },
26    { time: "15:00", status: "low" as const },
27    { time: "16:00", status: "available" as const },
28  ];
29
30  const emptyDelivery: SavedDeliveryAddress = { line1: "", line2: "", city: "", postcode: "", phone: "" };
31
32  function digitsOnly(s: string) {
33    return s.replace(/D/g, "");
34  }
35
36  function last4fromCardInput(cardNumber: string) {
37    const d = digitsOnly(cardNumber);
38    return d.slice(-4).padStart(4, "0").slice(-4);
39  }
40
41  export default function Checkout() {
42    const {
43      items,
44      subtotal,
45      pointsToRedeem,
46      setPointsToRedeem,
47      loyaltyDiscount,
48      totalToPay,
49    } = useCart();
50  }
```

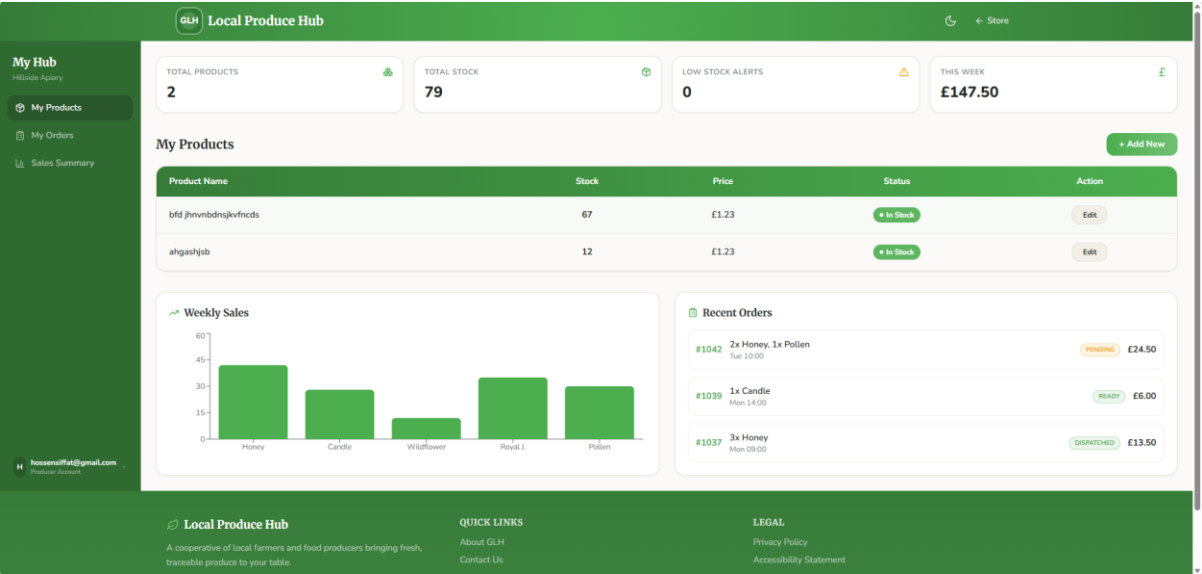

Producer Dashboard Page

The producer dashboard is the main management area for producer accounts. From here, producers can add new products to their listings, update stock levels, view a product table with stats, and access their producer-specific account settings.

An important fix was made during development to ensure that product changes persist in the backend database. Before this fix, any updates a producer made to their products would be lost after a page refresh because the data was only being held in temporary state. Now all changes are written to the PostgreSQL database, so they remain correctly in place after a refresh.

This page is implemented in `src/pages/ProducerDashboard.tsx`.

Screenshot Producer Dashboard



Screenshot Add Product / Update Stock

My Products + Add New

Product Name	Stock	Price	Status	Action
bfd jhvnbnjdnsjvfneds	67	£1.23	In Stock	Save
ahgashjsjb	12	£1.23	In Stock	Edit

Screenshot Producer Dashboard Code (src/pages/ProducerDashboard.tsx)

```
1 import { useState, useRef, useEffect } from "react";
2 import { Link, useNavigate } from "react-router-dom";
3 import StockBadge from "@components/StockBadge";
4 import Footer from "@components/Footer";
5 import ThemeToggle from "@components/ThemeToggle";
6 import { BarChart, Bar, XAxis, YAxis, ResponsiveContainer, Tooltip } from "recharts";
7 import { Package, ClipboardList, BarChart3, TrendingUp, AlertTriangle, Boxes, PoundSterling, ArrowLeft, Logout, ChevronsUp, Settings, X } from "lucide-react";
8 import { useAuth } from "@context/AuthContext";
9 import AccountSettings from "@components/dashboard/AccountSettings";
10 import { toast } from "sonner";
11 import {
12   createMyProduct,
13   getMyProducts,
14   updateMyProductStock,
15 } from "@lib/api/products";
16
17 const logo = "/favicon-glh.svg";
18 interface DashboardProduct {
19   id: string;
20   name: string;
21   stock: number;
22   price: number;
23 }
24
25 const salesData = [
26   { name: "Honey", sales: 42 },
27   { name: "Candle", sales: 28 },
28   { name: "Wildflower", sales: 12 },
29   { name: "Royal J.", sales: 35 },
30   { name: "Pollen", sales: 30 },
31 ];
32
33 const recentOrders = [
34   { id: "#1842", items: "2x Honey, 1x Pollen", date: "Tue 18:00", total: "£24.50", status: "Pending" },
35   { id: "#1839", items: "1x Candle", date: "Mon 14:00", total: "£6.00", status: "Ready" },
36   { id: "#1837", items: "3x Honey", date: "Mon 09:00", total: "£13.50", status: "Dispatched" },
37 ];
38
39 type Tab = "products" | "orders" | "sales" | "settings";
40
41 export default function ProducerDashboard() {
42   const [activeTab, setActiveTab] = useState<Tab>("products");
43   const [dashProducts, setDashProducts] = useState<DashboardProduct[]>([]);
44   const [editingId, setEditingId] = useState<string | null>(null);
45   const [editStock, setEditStock] = useState("");
46   const [showAccountMenu, setShowAccountMenu] = useState(false);
47   const accountMenuRef = useRef<HTMLDivElement>(null);
48   const { user, signOut } = useAuth();
```

Accessibility Panel / Accessibility Features

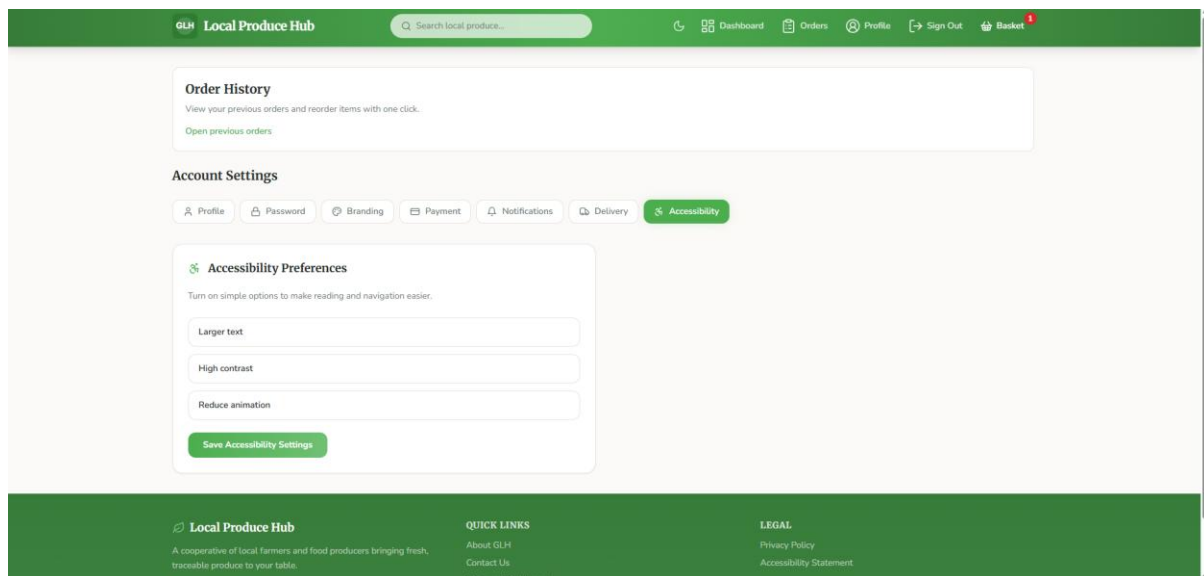
Accessibility features were identified in the client brief as a requirement to support a wide range of users. I implemented these through the account settings area, which made it more easily accessible for users in need.

The three accessibility options available are:

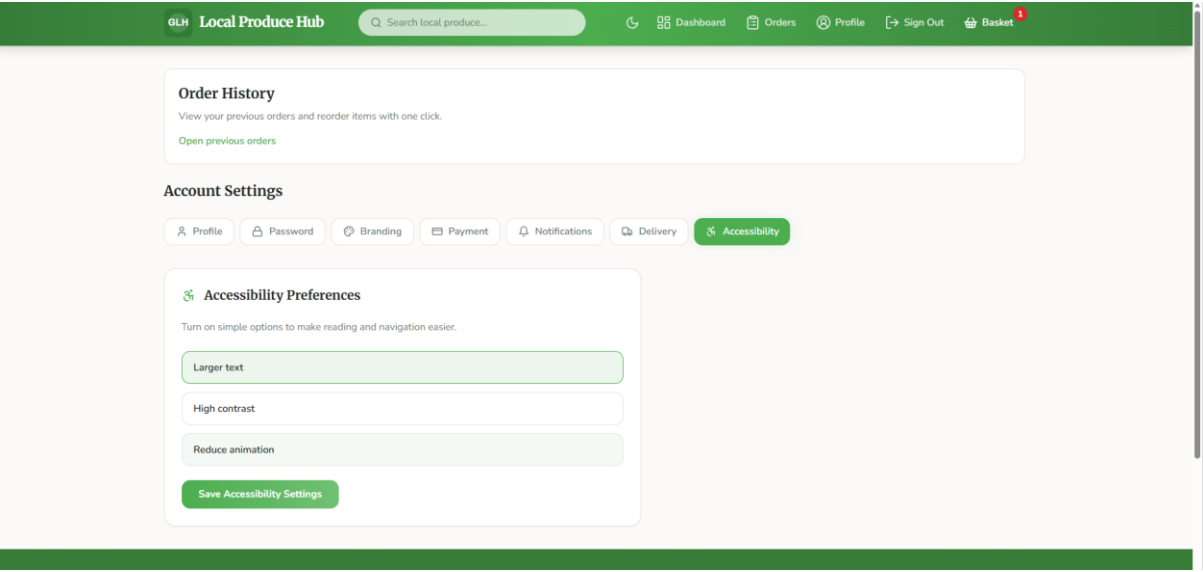
- Larger text - increases font sizes across the site for improved readability
- High contrast - applies a higher contrast colour scheme to improve visibility
- Reduced motion - disables animations and transitions for users who may be affected by movement on screen

These preferences can be saved to the user's account and are applied to the whole website when they are logged in. The functionality is handled across `src/components/dashboard/AccountSettings.tsx`, `src/lib/accessibilityPrefs.ts`, and `src/index.css`.

Screenshot Accessibility Settings Panel



Screenshot Accessibility Applied



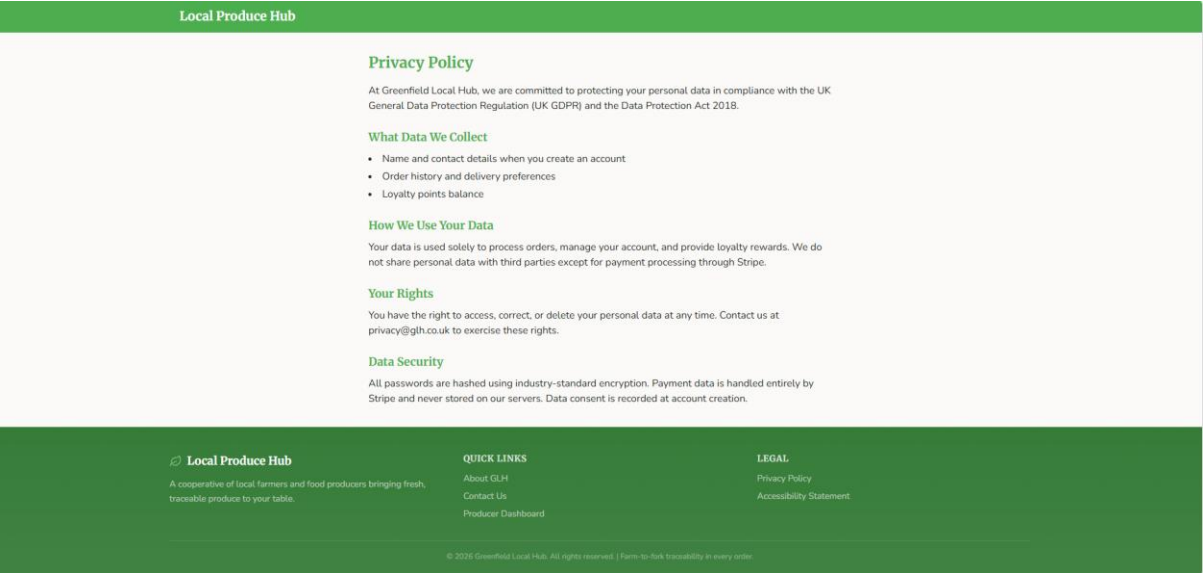
Screenshot Accessibility Code

```
1  const STORAGE_KEY = "ghc_accessibility_prefs_v1";
2
3  export interface AccessibilityPrefs {
4    largeText: boolean;
5    highContrast: boolean;
6    reduceMotion: boolean;
7  }
8
9  const defaults: AccessibilityPrefs = {
10    largeText: false,
11    highContrast: false,
12    reduceMotion: false,
13  };
14
15  export function loadAccessibilityPrefs(): AccessibilityPrefs {
16    if (typeof window === "undefined") return defaults;
17    try {
18      const raw = window.localStorage.getItem(STORAGE_KEY);
19      if (!raw) return defaults;
20      const parsed = JSON.parse(raw) as Partial<AccessibilityPrefs>;
21      return {
22        largeText: Boolean(parsed.largeText),
23        highContrast: Boolean(parsed.highContrast),
24        reduceMotion: Boolean(parsed.reduceMotion),
25      };
26    } catch {
27      return defaults;
28    }
29  }
30
31  export function saveAccessibilityPrefs(prefs: AccessibilityPrefs) {
32    if (typeof window === "undefined") return;
33    window.localStorage.setItem(STORAGE_KEY, JSON.stringify(prefs));
34  }
35
36  export function applyAccessibilityPrefs(prefs: AccessibilityPrefs) {
37    if (typeof document === "undefined") return;
38    const root = document.documentElement;
39    root.classList.toggle("ally-large-text", prefs.largeText);
40    root.classList.toggle("ally-high-contrast", prefs.highContrast);
41    root.classList.toggle("ally-reduce-motion", prefs.reduceMotion);
42  }
43
```

Policy Page

I also included a privacy and policy page in this prototype to inform and allow users to know about how their data is being handled. This is important for building trust with both customers and producers using the platform. This page is implemented in `src/pages/Privacy.tsx`.

Screenshot Policy Page



Screenshot — Policy Page Code (src/pages/Privacy.tsx)

```
src > pages > Privacy.tsx > ...
1 import { Link } from "react-router-dom";
2 import Footer from "@components/Footer";
3
4 export default function Privacy() {
5   return (
6     <div className="flex flex-col min-h-screen">
7       <nav className="bg-primary px-4 py-3">
8         <div className="container mx-auto">
9           <Link to="/" className="text-primary-foreground font-serif text-xl font-bold">
10             Local Produce Hub
11           </Link>
12         </div>
13       </nav>
14       <main className="container mx-auto flex-1 max-w-3xl px-4 py-8">
15         <h1 className="text-2xl font-bold font-serif text-primary mb-4">Privacy Policy</h1>
16         <p className="mb-4">
17           At Greenfield Local Hub, we are committed to protecting your personal data in compliance with the UK General Data Protection Regulation (UK GDPR) and the Data Protection Act 2018.
18         </p>
19         <h2 className="text-lg font-bold font-serif text-primary mt-6 mb-2">What Data We Collect</h2>
20         <ul className="list-disc list-inside space-y-1 mb-4">
21           <li>Name and contact details when you create an account</li>
22           <li>Order history and delivery preferences</li>
23           <li>Loyalty points balance</li>
24         </ul>
25         <h2 className="text-lg font-bold font-serif text-primary mt-6 mb-2">How We Use Your Data</h2>
26         <p className="mb-4">Your data is used solely to process orders, manage your account, and provide loyalty rewards. We do not share personal data with third parties except for payment processing through Stripe.
27       </p>
28         <h2 className="text-lg font-bold font-serif text-primary mt-6 mb-2">Your Rights</h2>
29         <p className="mb-4">You have the right to access, correct, or delete your personal data at any time. Contact us at privacy@glh.co.uk to exercise these rights.</p>
30         <h2 className="text-lg font-bold font-serif text-primary mt-6 mb-2">Data Security</h2>
31         <p>All passwords are hashed using industry-standard encryption. Payment data is handled entirely by Stripe and never stored on our servers. Data consent is recorded at account creation.</p>
32       </main>
33       <Footer />
34     </div>
35   );
36 }
```

Additional Notes

There were several technical challenges I have faced during this development that required significant time to resolve. Documenting these is useful as a record of decisions made and as a reference for future development.

Key issues resolved during development:

- Route and proxy errors that were returning HTML responses instead of JSON, these caused the frontend to break when making API calls and required careful debugging of the Express routing configuration.
- The reorder process and cart state updates were not functioning correctly, items from a previous order were not being added to the basket properly, which was resolved by fixing how the cart state is updated on reorder.
- Items Ordered were not saving into the order history, this was caused by the item lines not being saved to the database at the point of order. The checkout process was updated to store the full item list correctly.
- Users bank and address profile data was not being stored per account, this was fixed by associating this data correctly with the logged in user's account in the database.
- The method of delivery was not being saved with the order and as a result, the order tracker was displaying incorrect statuses. This was resolved by including the method of delivery in the order record at the time of creation.

All of these problems were resolved through iterative development and testing. Overall, the prototype meets the key functional requirements set out in the GLH client brief and provides a solid foundation for future development.